

Resilience

A hands-on course in building agents by building their harness

Lesson 8

Everything so far assumed the other side answers

Lesson 7 can tell you exactly where a run broke. Over a long run, this is what breaks first:

- the API drops a call, or the model is overloaded
- a response is cut off in the middle of a command
- the process dies halfway through, and the work goes with it
- or you stop it yourself, with Lesson 6's ESC, and what it had already done goes with it

Keep going, or stop somewhere you can start again

- **Get through the failures that pass**
- **Leave a state you can pick up from** when one doesn't, so the hour before it isn't lost

Built on the **model interface** and **output**, where the harness reaches out and the world gets to say no, and where work is in flight when you press ESC. To pick a run back up it **reads context's own record**: Lesson 4's episode.

Not every failure means the same thing

Worth trying again	Not worth trying again
the network dropped, or a timeout	a bad API key (401)
"slow down" (429)	a request the API won't accept (400)
any 5xx, and "overloaded" (529)	a model that doesn't exist (404)

The SDK already retries the left column, with backoff and jitter (`max_retries`).
What it can't do is choose a different model: that's what quark adds.

Every request gets an answer the model can read

A command can fail in ways the harness doesn't see coming:

- it hangs (Lesson 5's sandbox already puts a time limit on that)
- it prints bytes that aren't text
- the request arrives cut off by the token limit, half a command

This was cut off before it was whole, and it never reached the world is something the model can act on. A crash isn't.

When you press ESC, keep what was in flight

Lesson 6 stops at once by throwing the partials away. But the words were written, and the command did its first steps in the world.

- What it had said goes into working memory in **whole blocks**
- Text that had started, and thinking that had finished and been **signed**
- A tool request it had begun: [your doing never reached the world]
- A stopped command keeps what it printed, then [your doing stopped before done]

The trap: a command that may already have run

- The episode has the model's request on disk *before* the command runs
- If the harness died mid-command, the record ends with a request and no result
- The API won't accept that, and the harness can't just run it again: it might have been `git push`
- So the missing result becomes an error: *interrupted; may or may not have run; check before repeating it*
- The model, which knows what the command was for, looks and decides

`call()` grows a backup

Lesson 7's `quark.py` plus 31 lines. `client = Anthropic(timeout=300, max_retries=3)`, a list of `MODELS`, and `class Down`; then (abridged):

```
def call(each=lambda event: None, **request):
    for model in MODELS:
        try:
            with client.messages.stream(model=model, **request) as stream:
                for event in stream:
                    if each(event): return stream.current_message_snapshot
                return stream.get_final_message()
        except (APIConnectionError, APIStatusError) as e:
            if ... e.status_code < 500 and e.status_code != 429: raise
            trace(event="model_failed", model=model, error=type(e).__name__)
    raise Down()
```

`unfinished()` reads the last episode back

In `# — context —`, after `add()` (abridged):

```
def unfinished():
    episodes = sorted(glob.glob(".quark/episodes/*.jsonl"))
    if not episodes: return None
    messages = [json.loads(line) for line in open(episodes[-1])]
    last = messages[-1]
    if last["role"] == "assistant" and not any(... "tool_use" ...): return None
    if read(f"unfinished session: ... pick it up? [y/N] ").lower() != "y": return None
    return episodes[-1], messages
```

In `# — input —`, it's asked before anything else is read:

```
resumed = unfinished()
input = "" if resumed else " ".join(sys.argv[1:]) or read("> ")
```

A resumed session continues in its own episode

In `# — control flow —` (abridged):

```
if resumed:
    EPISODE, working_memory = resumed
    if working_memory[-1]["role"] == "assistant":
        add(working_memory, {"role": "user", "content": [...]}))
else:
    add(working_memory, {"role": "user", "content": input})
trace(event="start", input=input, resumed=bool(resumed))
```

Each unanswered request gets: *"interrupted: the harness stopped before this finished, so it may or may not have run. Check before repeating it."*

ESC: keep what it had said and printed

While it thinks or says (abridged):

```
if response.stop_reason is None:
    trace(event="interrupted", during="saying")
    print()
    kept = [b for b in output if (b.type != "text" or b.text)
            and (b.type != "thinking" or b.signature)]
    if kept: add(working_memory, {"role": "assistant", "content": kept})
    add(working_memory, {"role": "user", "content": [...] + [...SAYING]})
    continue
```

While it acts, Lesson 6's `=` becomes `+=` (abridged):

```
if ESC.is_set(): done.stdout += "\n[your doing stopped before done]"
```

Stop with a sentence, and never run half a command

```
except Down:
    sys.exit("[the model isn't answering. ...]")
...
    cmd = block.input.get("cmd")
    print(f"$ {cmd}")
    ...
    if not cmd or (response.stop_reason == "max_tokens" and ...):
        print("[cut off, not run]")
        input.append({"type": "tool_result", ..., "content": "[... cut off ...]"})
        continue
```

(abridged) The model is told *"your doing was cut off before it was fully formed — it never reached the world"*. And `errors="replace"` turns bytes that aren't text into



Run it: the preferred model is down

A stand-in for the API answers "529 overloaded" to every request for `claude-sonnet-5-5`, and forwards the rest:

```
$ wc -c notes.txt
6 notes.txt

notes.txt is 6 bytes.
```

The output looks like any other run. The trace doesn't (shortened):

```
{"event":"model_failed","model":"claude-sonnet-5-5","error":"OverloadedError"}
{"event":"model","seconds":5.27,"stop_reason":"tool_use"}
```

Run it: nothing answers, then the API comes back

Every request fails, for every model. quark stops with:

[the model isn't answering. Everything so far is in the episode; run quark again to pick it up]

Run `quark.py` again with no input and answer `y` (shortened):

```
6 notes.txt
```

```
notes.txt is 6 bytes.
```

```
{"event": "start", "input": "", "resumed": true}
```

Run it: killed mid-command, then resumed

`sleep 15 && echo finished > flag.txt` , killed with `kill -9` on its PID twelve seconds in. The container wasn't, so the command finished. Resumed (shortened):

```
-rw-rw-rw- 1 root root 9 Oct  6 20:33 flag.txt  
Tue Oct  6 20:33:44 UTC 2026
```

```
`flag.txt` contains `finished`.
```

The harness reported my first run as interrupted, so I checked the file before running the command again. It was already there with that content, which means the command had completed. I didn't run it a second time.

Run it: ESC, and the model knows how far it got

While saying, the second answer picks up where the first stopped (shortened):

The same trick works for any prime where $10^6 \equiv 1$: the remainder is always 4, so none
Sorry, I was cut off partway through. Here is where things stand.

While doing, a line a second, ESC after three and a half (shortened):

step 4

[your doing stopped before done]
You interrupted the command, so it stopped after step 4. Steps 5 through 10 never ran.

Run it: every request cut off, nothing run

A copy with `max_tokens` lowered to 400, and a 40-line poem to write in one command (shortened):

```
$ None  
[cut off, not run]  
$ None  
[cut off, not run]
```

- Each request arrived with no `cmd` at all, so the line says `$ None`
- Each was answered with the cut-off result; none of them ran
- After the eighth, the model stopped and said what was happening, with nothing half-written on disk

What else resilience can be

- **Waiting and the backup:** backoff with a cap, `Retry-After`, a limiter; another model, provider or region
- **Remembering a failure:** a *circuit breaker* leaves a failed model alone for a minute
- **A broken stream:** keep what arrived, as ESC does; time out on *silence*, not the whole reply
- **What's kept when stopped:** unsigned thinking as plain text, a half-written command as text
- **Tools, state, restarts:** kill the process group, cap output; save every step; restart by hand or by a supervisor

Products, and a fuller example

Gateways like LiteLLM and OpenRouter handle retries, fallbacks and rate limits. Durable execution (Temporal) and checkpointing (LangGraph) pick a crashed process up at its step.

`resilience.py`, built on Lesson 3's loop, does the retries itself, in plain sight:

- `retryable()` sorts failures; `pause()` honors `retry-after`, else backs off with jitter
- `ask()` benches a model that gave up for `BENCH` seconds
- `run()` kills the whole process group and caps output
- `save()` checkpoints every step; `resilience.py resume` picks it up

The preferred model down, with a circuit breaker

```
[claude-sonnet-5-5: OverloadedError, try 1 of 4, waiting 0.7s]
[claude-sonnet-5-5: OverloadedError, try 2 of 4, waiting 1.7s]
[claude-sonnet-5-5: OverloadedError, try 3 of 4, waiting 2.5s]
[claude-sonnet-5-5: OverloadedError, giving up on it for 60s]
[answered by the backup, claude-opus-5-5]
$ wc -c notes.txt
6 notes.txt
[answered by the backup, claude-opus-5-5]
notes.txt is 6 bytes.
[done in 2 steps]
```

Waits grow with jitter: 0.7, 1.7, 2.5. The second call skipped the benched model: six requests, where quark sent ten.

The rule

Assume the call can fail, and make failure something the harness handles instead of something that ends the run.

- Retry what passes, a bounded number of times; have somewhere else to go
- Stop with a sentence when everywhere is out
- Answer every tool request, even a broken one, with a result the model can read
- When you stop it, keep what it had already said and done
- Keep a record written before each step, so what was in flight is reported as *unknown*

What resilience never does

- **Context:** it reads the record context already keeps; it adds no store of its own
- **Control flow:** same loop, same stops; a retry is inside one call, and the only new way out is `Down`
- **Input:** untouched, apart from one question at startup, asked with the same `read()`
- **What the model sees:** only what it's told about its own work: the interrupted and cut-off results, and the partials ESC used to throw away

What's missing: alive, but not fast or cheap

- Every call re-sends the whole conversation; only the system prompt is cached
- A command that prints a megabyte puts a megabyte in the next request
- Three commands that could run side by side wait for each other
- The summary is written by the same big model as everything else

Making each step smaller, faster and cheaper is its own layer.

Next: Performance

Lesson 9